

ASP.NET MVC in VS Code

Visual Studio vs. VS Code Transition Note

In VS Code, terminal CLI commands replace Visual Studio's GUI wizards. All core C# code, MVC logic, Razor views, and Entity Framework syntax remain 100% identical.

1. PROJECT & SOLUTION CREATION

Task / Description	Command
Create ASP.NET MVC Project	<code>dotnet new mvc -n ProjectName</code>
Create Web API Project	<code>dotnet new webapi -n ProjectName</code>
Create Class Library	<code>dotnet new classlib -n ProjectName</code>
Create Empty Solution (.sln)	<code>dotnet new sln -n SolutionName</code>
Add Project to Solution	<code>dotnet sln add Path/To/Project.csproj</code>

2. MANAGING REFERENCES & PACKAGES

Task / Description	Command
Add Project Reference (Link App & ClassLib)	<code>dotnet add ProjectA.csproj reference Path/To/ProjectB.csproj</code>
Remove Project Reference	<code>dotnet remove ProjectA.csproj reference Path/To/ProjectB.csproj</code>
Install NuGet Package	<code>dotnet add package PackageName</code>
Uninstall NuGet Package	<code>dotnet remove package PackageName</code>

3. INDIVIDUAL FILE CREATION

Task / Description	Command
Create C# Class File	<code>dotnet new class -n MyClassName</code>
Create View / CSHTML (Windows)	<code>New-Item -Path "Views/Home/About.cshtml" -ItemType File</code>
Create View / CSHTML (Mac / Linux / Git Bash)	<code>touch Views/Home/About.cshtml</code>

4. BUILD, RUN, AND DEBUG

Task / Description	Command

5. CODE GENERATOR TOOL (VISUAL STUDIO SCAFFOLDING)

To auto-generate complete Controllers and Razor Views (like Visual Studio's right-click menu), install the scaffolding tool and required package once per project:

Step 1: Install Tool Globally (One time)

dotnet tool install -g dotnet-aspnet-codegenerator

Step 2: Add Package to Project

dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Tools

Scaffold Target	Command
Empty Controller	dotnet-aspnet-codegenerator controller -name ProductController -outDir Contro
CRUD Controller with EF Context	dotnet-aspnet-codegenerator controller -name ProductController -m Product -dc
Empty Razor View (.cshtml)	dotnet-aspnet-codegenerator view Index Empty -outDir Views/Product
CRUD Razor View (Create/Edit/List)	dotnet-aspnet-codegenerator view Create Create -m Product -outDir Views/Produ

6. COMPLETE MULTI-PROJECT SOLUTION WALKTHROUGH

Step-by-step terminal execution to build a standard enterprise architecture (Solution + Web App + Class Library):

```
# 1. Create a root directory and enter it
mkdir MyEnterpriseApp && cd MyEnterpriseApp

# 2. Create the main Solution file
dotnet new sln -n MyEnterpriseApp

# 3. Create the MVC Web Application project
dotnet new mvc -n MyApp.Web

# 4. Create a Class Library for Business/Data Logic
dotnet new classlib -n MyApp.Core

# 5. Add both projects to the Solution
dotnet sln add MyApp.Web/MyApp.Web.csproj
dotnet sln add MyApp.Core/MyApp.Core.csproj

# 6. Reference the Core library inside the Web project
dotnet add MyApp.Web/MyApp.Web.csproj reference MyApp.Core/MyApp.Core.csproj

# 7. Restore dependencies and run the MVC app
cd MyApp.Web
dotnet watch
```

Pro Tip: Recommended VS Code Extensions

Install **C# Dev Kit** and **C# Namespace Autocompleter** from the VS Code Marketplace to enable Solution Explorer sidebar views and automatic namespace imports.